

MET CS688

WEB ANALYTICS AND MINING

ZLATKO VASILKOSKI

LOCAL LLMS

Running LLMs Locally

Why Use Local LLMs?

- Running large language models (LLMs) like GPT and Claude usually involves sending data to servers managed by OpenAI and other AI model providers.
- While these services are secure, some businesses prefer to keep their data entirely offline for greater privacy.
- These local tools do not collect user data or track users' actions when they use it to run local LLMs.

Advantages:

- **Privacy**: You can prompt local LLMs in a multi-turn manner without your prompt data leaving your localhost.
- **Customization Options**: Local LLMs provide advanced configurations for CPU threads, temperature, context length, GPU settings, and more. This is similar to OpenAI's playground.
- **Support and Security**: They provide similar support and security as OpenAI or Claude.
- **Subscription and Cost**: These tools are free to use and they do not require monthly subscription.
- **Offline Support**: You can load and connect with large language models while offline.
- **Connectivity**: Sometimes, connecting to a cloud service like OpenAI may result in poor signal and connection.



Which Tool Fits You Best?

Some of these tools are:

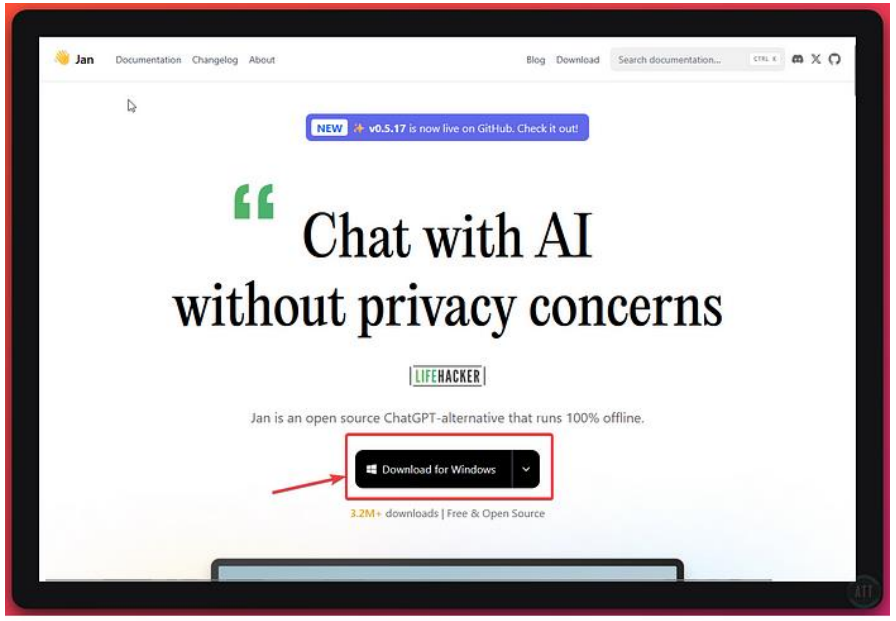
- **Jan (Jan.ai)**
  - Jan is an open-source imitation version of ChatGPT designed to operate offline. Runs several popular models.
- **GPT4ALL**
  - One of the largest number of users for this kind of tools.
  - The app collects anonymous user data about usage analytics and chat sharing.
- **LM Studio**
  - The local server provides sample Curl and Python client requests. This feature helps to build an AI application using LM Studio to access a particular LLM.
- **LLaMa.cpp**
  - Is the underlying backend technology (inference engine) that powers local LLM tools like Ollama and many others.
- **Ollama**
  - It has the largest number of contributors and is more extendable among the other open-source LLM tools of this kind.
- **Llamafile** (backed by Mozilla)
  - Unlike previous LLM tools (LM Studio and Jan), Llamafile requires only one executable file to run LLMs.
  - Gives the fastest prompt processing experience. Excellent option for summarizing long text and large documents.

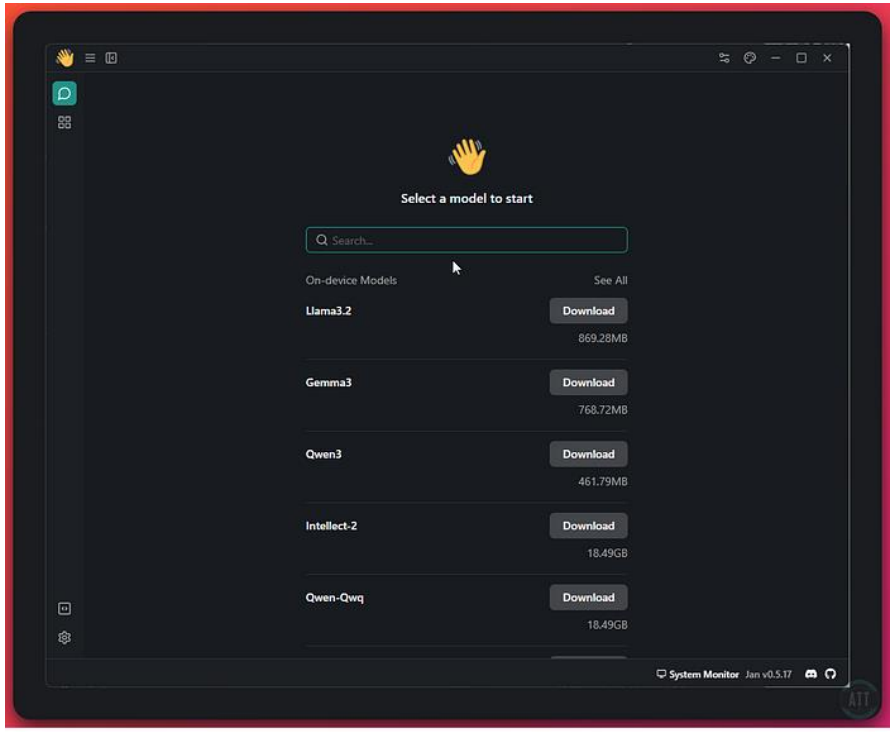
Tool	Fastest/Easiest	CPU Support	GPU Support	3b RAM (CPU/GPU)	7b RAM (CPU/GPU)	13b RAM (CPU/GPU)	OS Support	Notable Features
Jan.ai	Easiest	Yes	Yes	8GB/6GB	16GB/8GB	32GB/12GB	Win/Mac/Linux	GUI, auto model download
GPT4All	Easy	Yes	Yes	8GB/6GB	16GB/8GB	32GB/12GB	Win/Mac/Linux	GUI, chat interface
LM Studio	Easy	Yes	Yes	8GB/6GB	16GB/8GB	32GB/12GB	Win/Mac	GUI, model marketplace
llama.cpp	Fast (CLI)	Yes	Yes*	8GB/6GB	16GB/8GB	32GB/12GB	Win/Mac/Linux	CLI, quantization
Ollama	Fastest	Yes	Yes	8GB/6GB	16GB/8GB	32GB/12GB	Win/Mac/Linux	CLI, REST API, model hub

Jan - Installing

What is Jan (Jan.ai)?

- Jan.ai is a user-friendly, cross-platform app for running open-source LLMs (like Llama, Mistral, DeepSeek) with a simple GUI — no coding needed!
    - Use built-in tools for summarization, code, or brainstorming.
- **Download**: Go to Jan.ai and grab the installer for your OS (Windows, Mac, or Linux).
- **Pick a Model:**
  Browse the built-in list (Llama 3, Mistral, DeepSeek, and more). Select one that fits your needs.
- **Download the Model:**
  Click “Download” and wait for the model to load. (For a few year-old laptops use smaller 3b or 7b models. They are faster and use less RAM.) A few years old laptops, stick with 3b or 7b models.





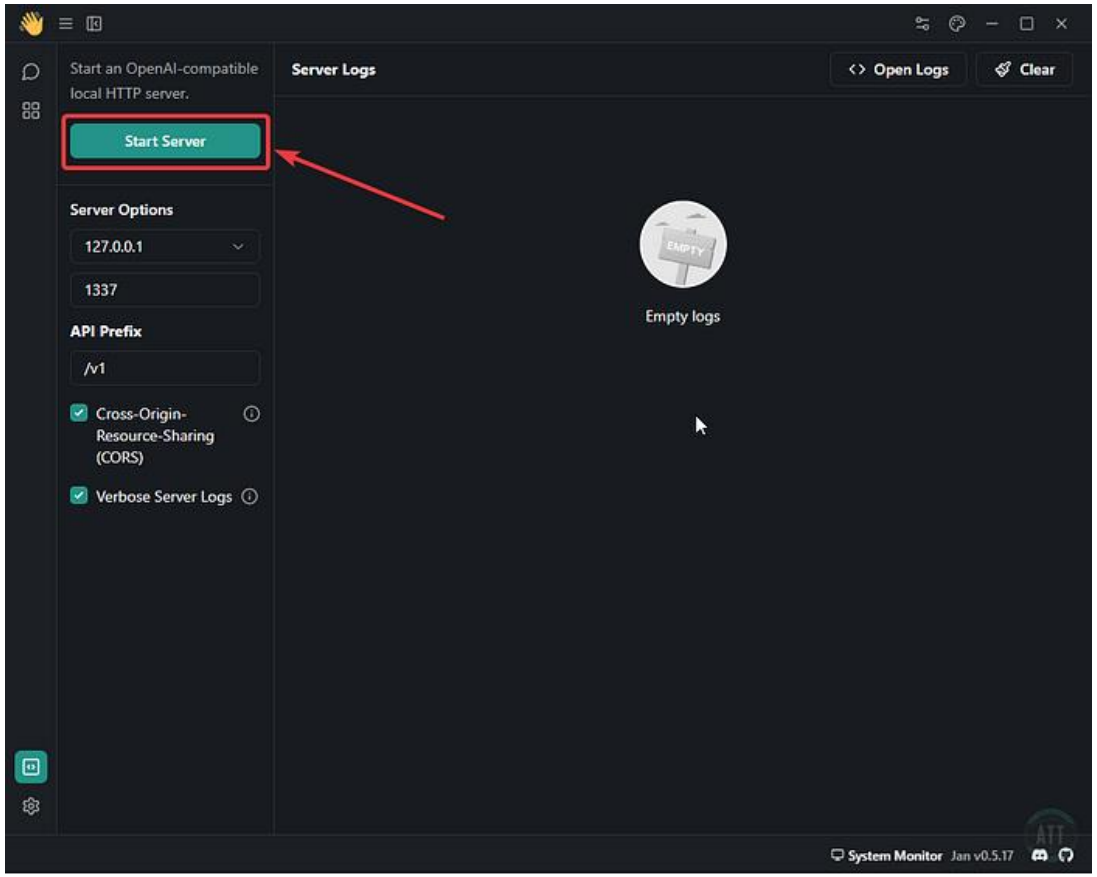
Jan - Examples & Projects

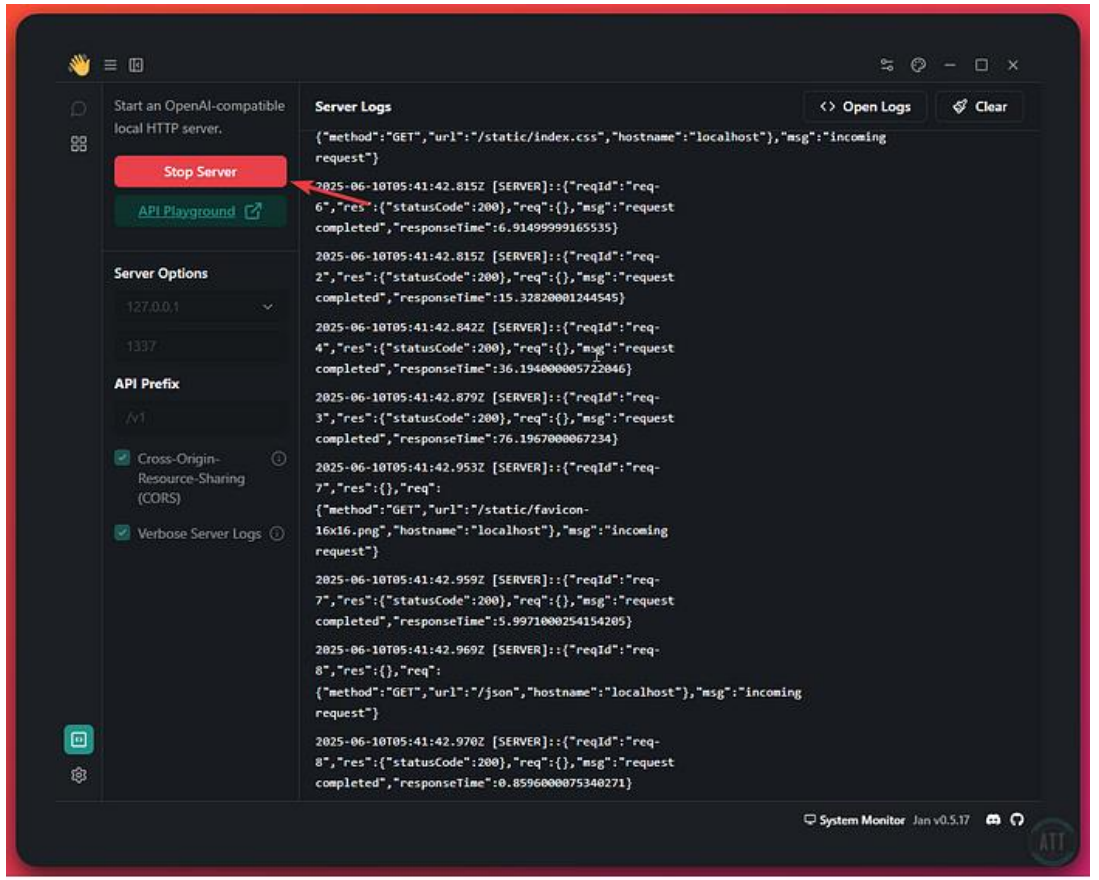
To start the local API server:

a. From the left side of the UI, select **Local API Server**.

b. Click the **Start Server** button.

c. To stop the server: Simply click the **Stop Server** button.





Jan - Examples & Projects

- **Code Example: Connecting to Jan.ai's Local API**
  - Before sending any prompts, you need to load the model using this endpoint:

```python
import requests

url = "http://127.0.0.1:1337/v1/models/start"

payload = { "model": "llama3.2:1b" }
headers = {"Content-Type": "application/json"}

response = requests.post(url, json=payload, headers=headers)

print(response.json())
```

- Send a Prompt to the Chat Completion Endpoint
  - This snippet sends a user prompt to a locally running instance of Jan.ai and prints the response.
- **Enable GPU Acceleration (Optional) :**
  If you have a compatible GPU, turn this on in settings for faster responses.

```python
url = "http://127.0.0.1:1337/v1/chat/completions"

payload = {
"messages": [
"role": "user",
"content": "Hey, can you tell me a joke?",
"name": "Anup"
},],
"model": "llama3.2:1b",
# "stream": True,
"max_tokens": 4096,

headers = {"Content-Type": "application/json"}

response = requests.post(url, json=payload, headers=headers)

print(response.json())
```

- **Best Practices & Tips**
  - Use smaller models (7B or less) for older laptops.
  - Check Jan's docs for troubleshooting and advanced features.
  - Try the "Assistant Instructions" feature to customize your AI's personality.
  - All processing happens offline — your data never leaves your laptop.

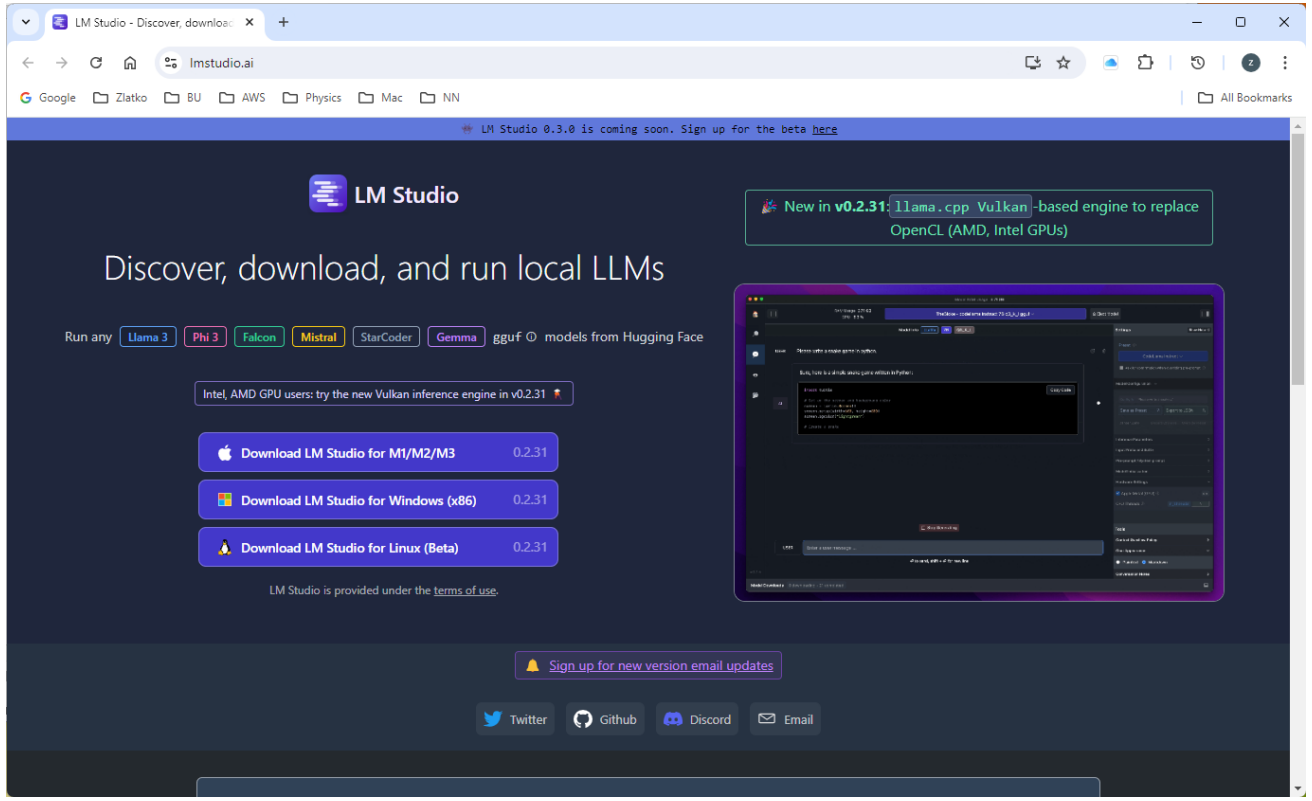
A

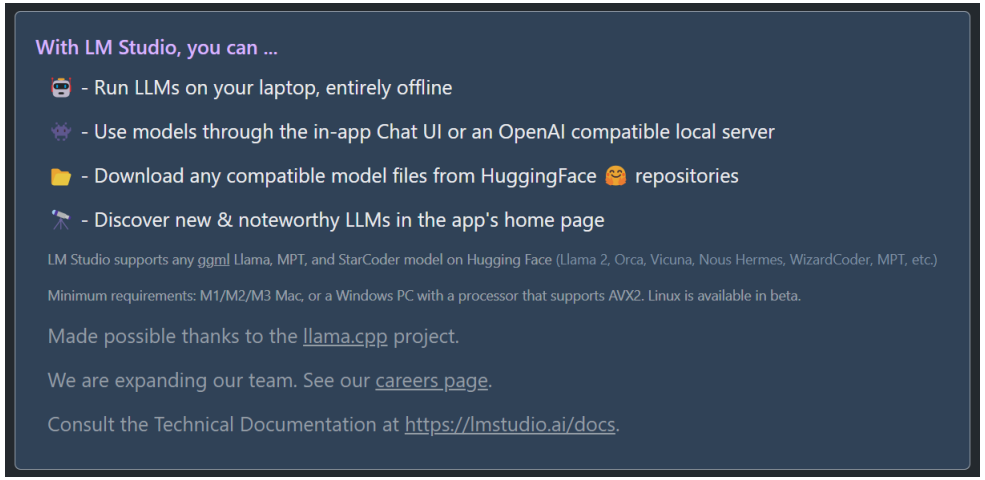
- A

LM Studio - Installing

What is LM Studio?

- LM Studio is an easy-to-use desktop app for experimenting with local and open-source Large Language Models (LLMs).
    - This downloadable software handles the complexities of running large language models on your local machine.
- **Download**: Install LM Studio for your operating system.
    - Link: https://lmstudio.ai/
- **Pick & Download a Model:**
  Pick a model (Llama 3, Mistral, etc.) from the curated list.
- **Chat, Code, or Analyze:**
  Use the built-in chat or code tools right in the app.
- **Local Inference Server:**
  Similar to Jan.ai, LM Studio allows you to customize your model and launch a local API server with just one click.
- **Using Multiple Models:**
  Ability to run and serve multiple models simultaneously.

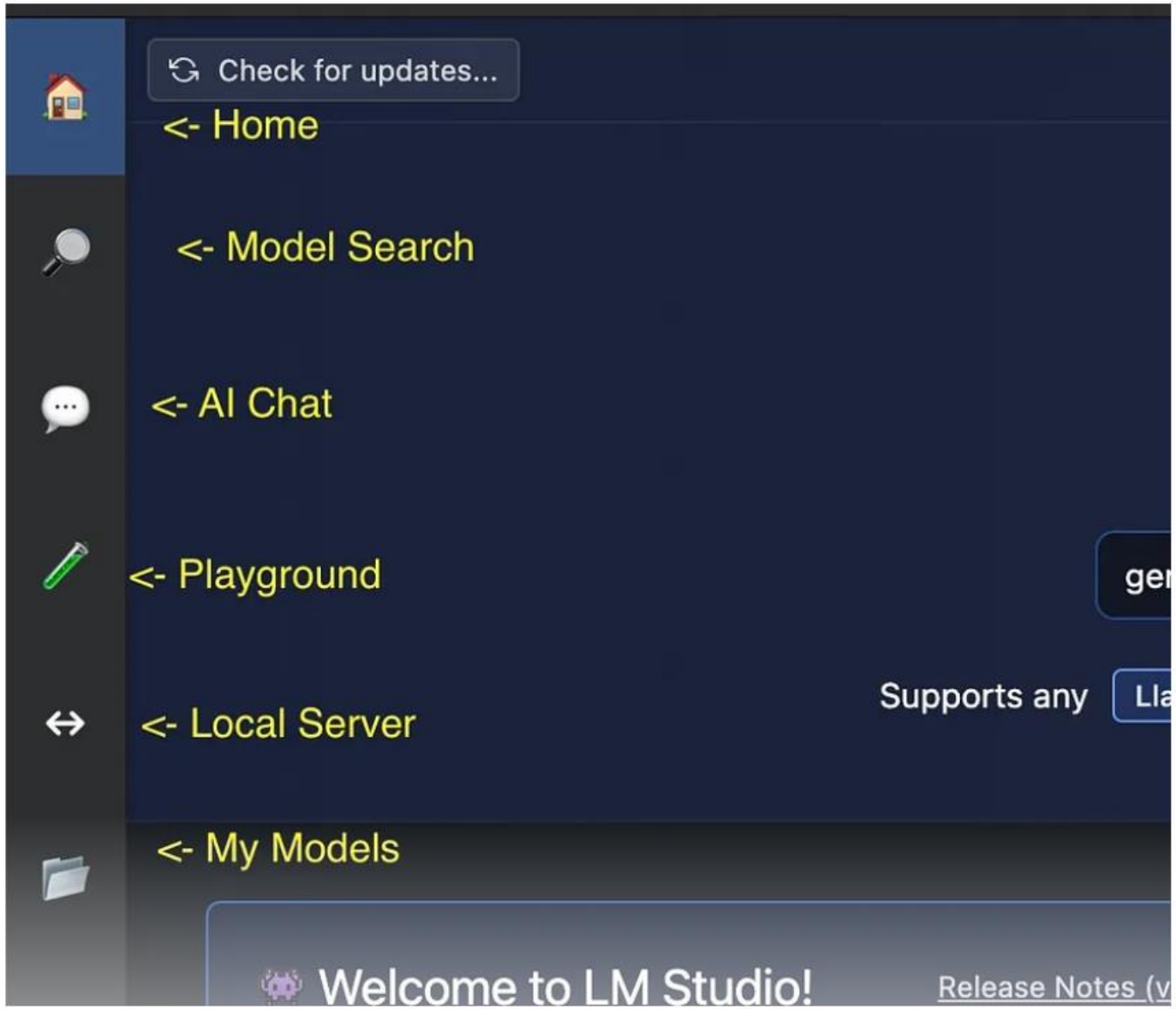




Exploring LM Studio

Explore tabs:

- **Model Search** – to find an LLM model
- **AI Chat** –Chat UI with your **local** model.
- Start **Local Server** to interact with Python with your LLM through LM Studio.
- **Playground** – start multimodal session.
- The key feature of LM Studio is that it offers the option to run and serve multiple models at once. This allows users to compare different model results and use them for multiple applications. In order to run multiple model sessions, you need a high GPU VRAM.
- **My Models** – See and choose your local LLM models.



Select an AI Model (LLM) to Download

Install LLM according to your computer capabilities and the task you want to achieve.

Larger AI models are more accurate but slowe and require more memory and computer resources.

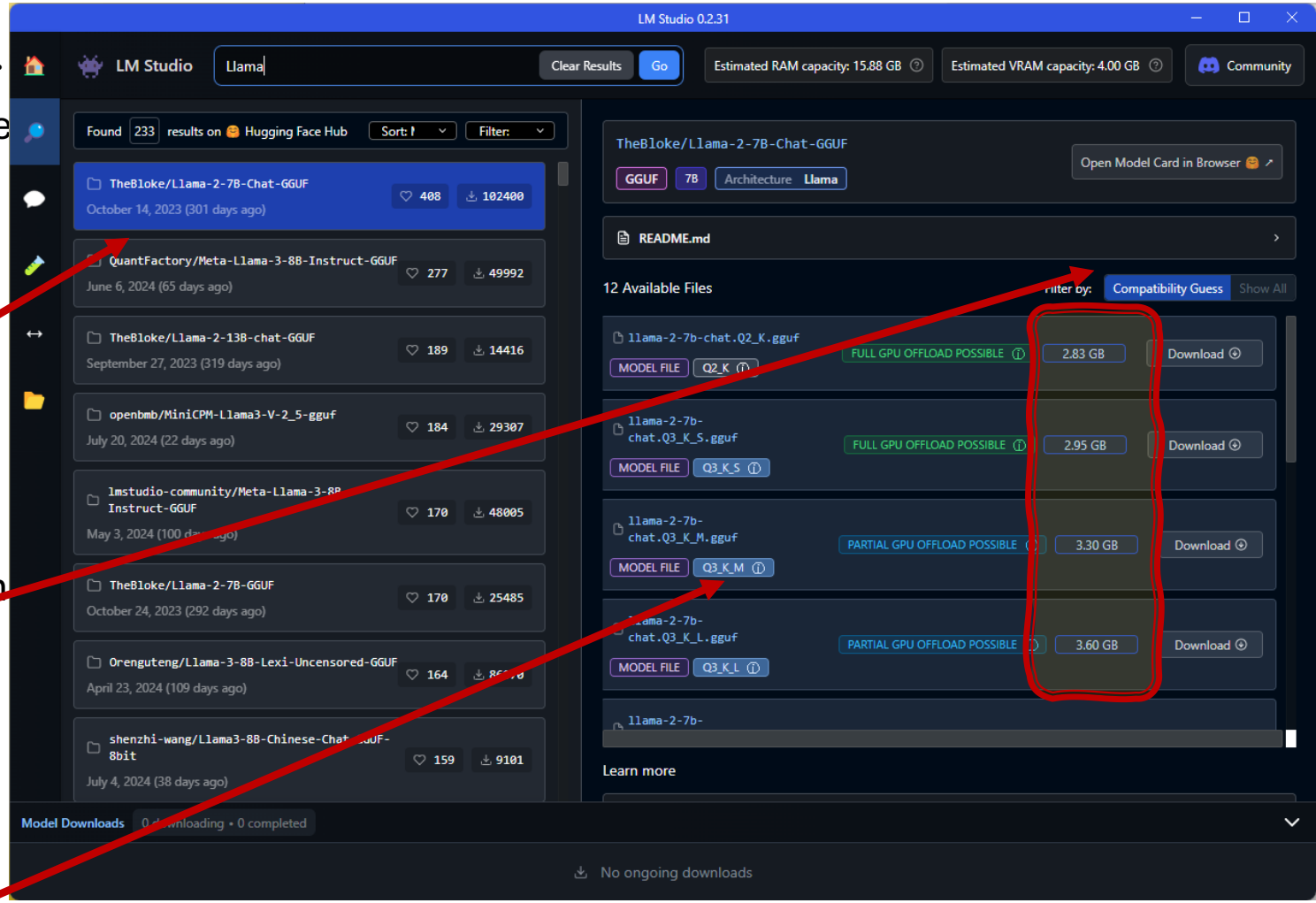
- Select one of the community suggested models listed in the main window.
- Use search bar for any model available at HuggingFace.

Select a model to download on the left. On th

right are the versions (sizes) of that model, selected by **Compatibility Guess**.

You can see the size of the install/download file. **So be sure you are okay with the size of the download.**

Most of these models are **quantized**.



What is Quantization?

- Quantization refers to using lower precision numbers like 8-bit integers rather than 32-bit floating point values to represent the weights and activations in the model.
- This reduces memory usage and speeds up inference on your computer's hardware.
- Quantization can reduce model accuracy slightly compared to a full precision version but provides up to 4x memory savings and faster inference.
- Think of it like how MP-3's are compressed music files or .jpgs are compressed image files.
- Although these are of less quality, you often won't see a significant difference.
- In the case of LLM's, the "Q" number you see in the listing of the LLM, represents the amount of quantization.
  - Lower number means more quantization and higher number means less quantization.
- The **GGML** and **GGUF** refer to the two quantization strategies
  - **GGML** or "Mixed Logits" - more flexible mixed-precision quantization framework, **higher compression rates**.
  - **GGUF** or "Uniformly Quantized Fully Connected" - is specifically optimized for uniformly quantizing all layers of Transformer models, offers **simpler deployment**.

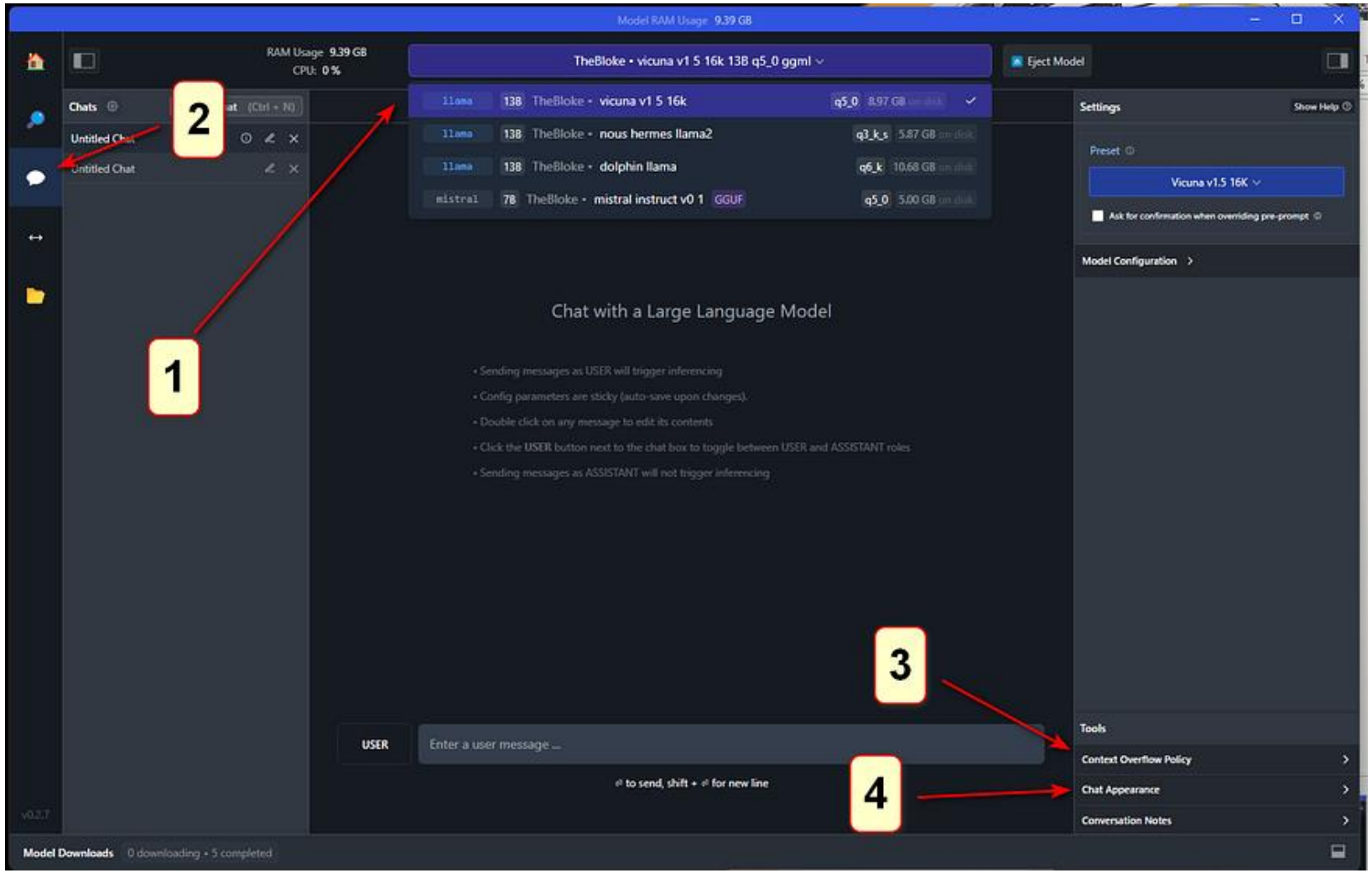
Select an LLM to install

Select one of the smallest AI models such as:

- Phi-2-layla-v1-chatml-gguf (1.1GB)
- Mistral-7B-Instruct-v0.3-GGUF (1.6GB)

Once the model has finished its download,

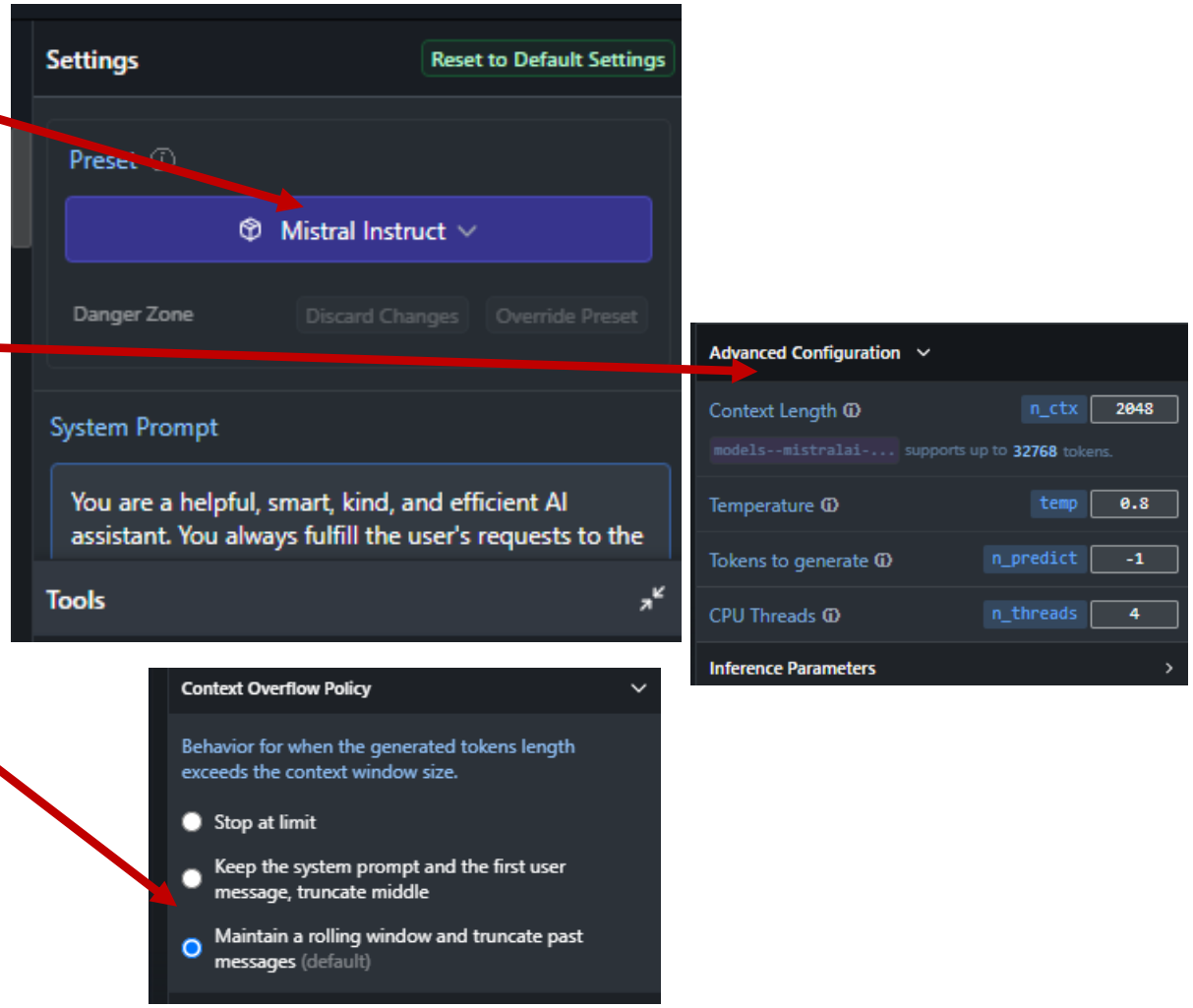
1. Select the model from the drop-down menu at the top of the window;
2. Select the **chat bubble** in the left side column;
3. Open up the following sections on the right, “**Context Overflow Policy**” and “**Chat Appearance**”.



Model Setup and Configuration

Select Preset for the model you want to use

- Set up model's role (**System Prompt**).
  - "You are comedian that specializes in jokes on science. Respond as Einstein would tell a joke."
- You can change model's parameters.
  - If you are not familiar with them, you can read their meaning by hovering over their names.
- You can set up GPU
- Make sure "Maintain a rolling window and truncate past messages" is selected under "Content Overflow Policy". This refers to what to do if the response is too long.
- Model's configuration can be saved and exported.



- When ready to use your local LLM. Simply go to AI Chat, enter your query in the "USER" field and the LLM will respond as "AI".

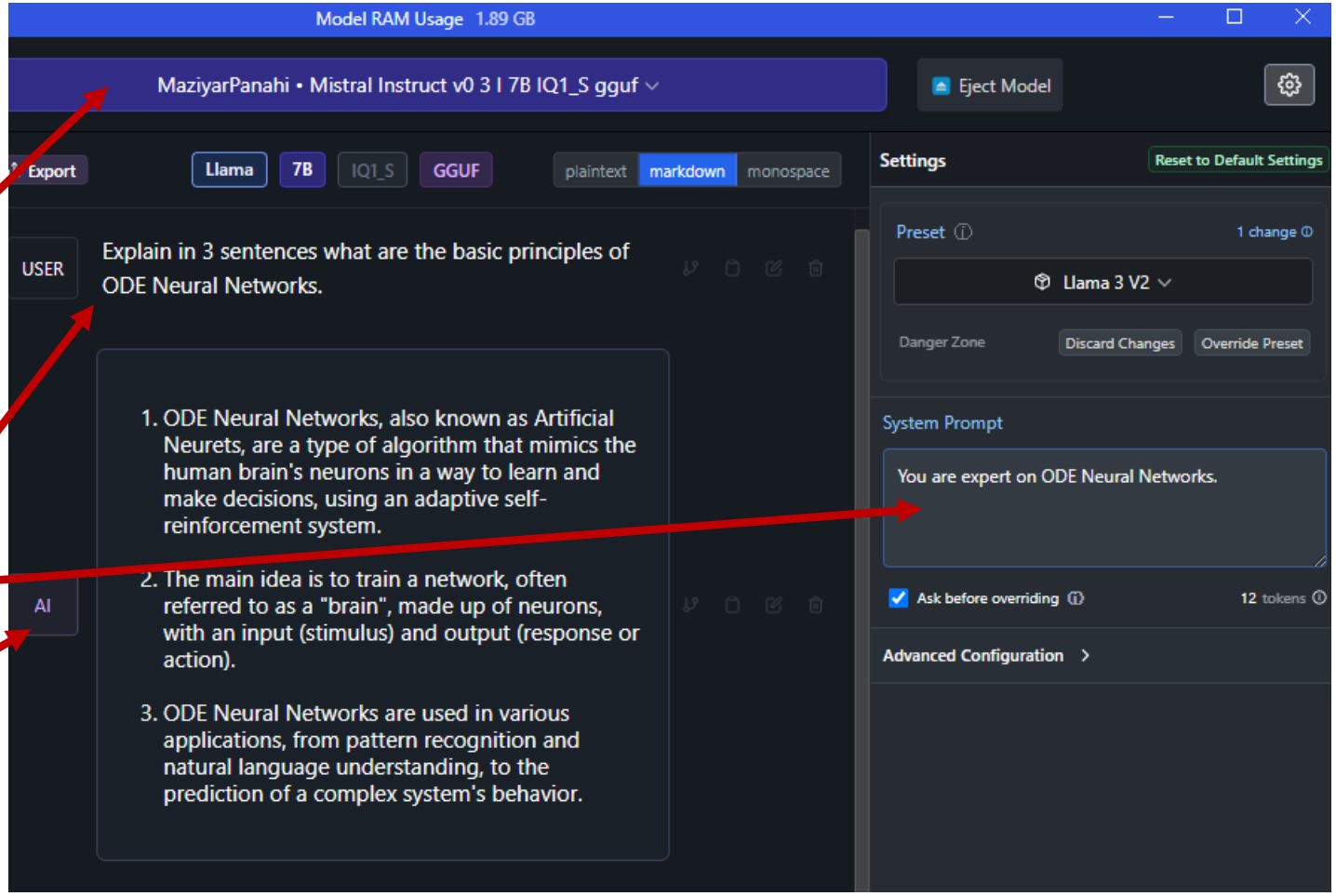
1) LM Studio Lab Project

Prerequisites:

- Download and setup an LLM (AI model) locally using LM Studio
- Load the LLM model.
- Give a role to your AI model by modifying the default prompt. For example:
  - "You are an expert on ODE Neural Networks."

Project 1 Task:

- Use LM Studio's **AI Chat** to start a chat session with your AI model (the LLM you downloaded locally)
  - Upload your local model
  - Set a role for your local AI model
  - Ask your local AI model questions
  - Generate a response to your question
  - Analyze and compare the quality of the answer with a Google search for example.



2) Project: LM Studio Integration with Python

Project 2 Task: Replicate Project 1 using Python. Integrate and access your local LLM with Python using LM Studio.

- Create a python script
- Copy example code from LM Studio in your script and integrate it as a function.
- Pass the same role and model that you used in Project 1 to get local LLM's reply from python
- Print the reply to compare it with Project 1.
- Integrate it into a Streamlit App.

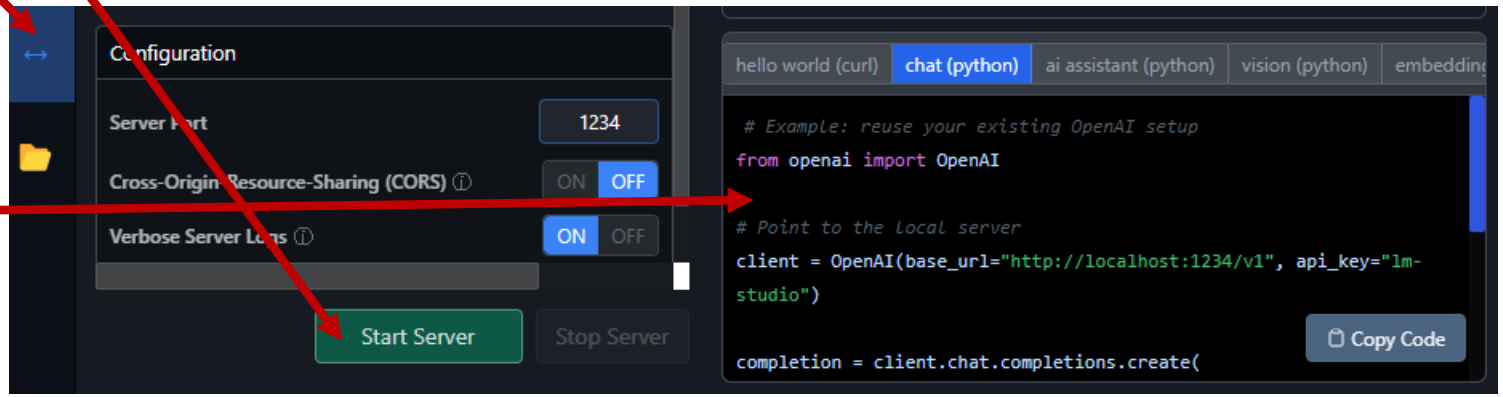
Project 3 Task: Create an AI Assistant in Python and chat with it using an LLM from LM Studio

- Integrate and access your local LLM with Python using LM Studio.
  - Keep responding to input questions with well-reasoned answers that "are both correct and helpful"
- Copy the ai.assistant example code from LM Studio to a python script & run it.

LM Studio Integration with Python

From Python you can use LM Studio as an interface to your downloaded LLMs

- Go to **Local Server** on your LM Studio
- Start LM Studio Server and change settings if you need to.
- This will start a local HTTP server that behaves as an OpenAI API endpoint that you can access from Python.
- An example client request (to insert into Python) is also given.
  - This includes your local base_url that you need to use instead of openai's, as well as the api_key which for local LLM you can set it up to anything.
- In this way you use all the benefits of the OpenAI library, but without the cost since you are using Open-Source local model!

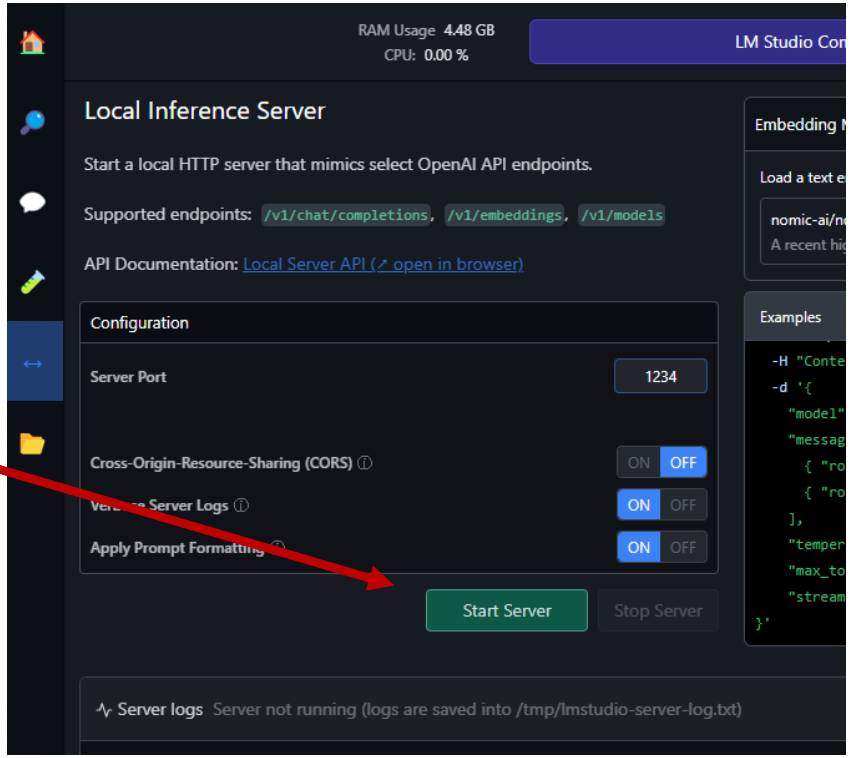


2) Use local LLM via LM Studio & Python

Project 2 Task: Replicate Project 1 using Python

- Load the LLM model to LM Studio
- Start LM Studio Server
- Copy Client Request code snippet example from LM Studio to Python

Note since you're using local LLM, no need to specify model & api_key




```python
# Example: reuse your existing OpenAI setup
from openai import OpenAI


1 usage
def get_lm_studio_reply(client):
    completion = client.chat.completions.create(
        model="local-model",
        messages=[
            {"role": "system", "content": "You are an expert on ODE Neural Networks."},
            {"role": "user",
             "content": "I want you to explain to me in 5 sentences the basic principles of ODE Neural Networks."}
        ],
        temperature=0.7,
    )
    return completion


if __name__ == '__main__':
    # Point to the local server
    client = OpenAI(base_url="http://localhost:1234/v1", api_key="lm-studio")
    answer = get_lm_studio_reply(client)
    print(answer.choices[0].message)
```

```
        "content": "Here's a brief explanation of the basic principles of ODE Neural Networks:\n\nODE (Ordinary Differential Equation) Neural Networks are a type of neural
network that leverages differential equations to model complex systems and behaviors. The core idea is to represent the dynamics of a system as an ODE, which defines how
the system changes over time, and then learn the parameters of this equation using data. This is achieved by optimizing the ODE's parameters to fit the observed behavior
of the system, typically using techniques like gradient descent or variational methods. The resulting ODE model can be used for tasks such as modeling and forecasting, as
well as controlling and designing complex systems. By incorporating physical knowledge and constraints into the ODE model, ODE Neural Networks can provide more
interpretable and accurate results compared to traditional neural network architectures."
    },
```

3) Local AI Assistant with LM Studio & Python

Project 3 Task:

- Create an AI Assistant in Python and chat with it using an LLM from LM Studio
  - Keep responding to input questions with well-reasoned answers that "are both correct and helpful”
- Copy the ai.assistant example code from LM Studio to a python script & run it.

```python
# Example: reuse your existing OpenAI setup
from openai import OpenAI

# Point to the local server
client = OpenAI(base_url="http://localhost:1234/v1", api_key="lm-studio")

history = [
    {"role": "system", "content": "You are intelligent assistant. You always provide well-reasoned answers that "
                                  "are both correct and helpful"},
    {"role": "user",
     "content": "Hello, introduce yourself to someone opening this program for the first time. Be concise."}
]

while True:
    completion = client.chat.completions.create(
        model="local-model",
        messages=history,
        temperature=0.7,
        stream=True,
    )

    new_message = {"role": "assistant", "content": ""}

    for chunk in completion:
        if chunk.choices[0].delta.content:
            print(chunk.choices[0].delta.content, end="", flush=True)
            new_message["content"] += chunk.choices[0].delta.content

    history.append(new_message)
```

```
Welcome! I'm LLaMA, an AI designed to assist and communicate with you in a natural and helpful way. I'll be your guide as we explore topics,
 answer questions, and have conversations. Let's get started!
> What is the capital of Macedonia?
The capital of North Macedonia (formerly known as the Republic of Macedonia) is Skopje.
> Who are the most prominent physicists involved in creating Quantum Loop Gravity?
Some of the most prominent physicists involved in developing quantum loop gravity (QLG) or related theories include:

1. Lee Smolin: Known for his work on quantum gravity, loop quantum gravity, and cosmology.
2. Carlo Rovelli: A pioneer of loop quantum gravity and its application to black holes and cosmology.
3. Abhay Ashtekar: Developed the theory of loop quantum gravity and its connection to spacetime geometry.
4. John Wheeler: Although not exclusively focused on QLG, he was a key figure in developing the concept of loops and knots in spacetime.
5. Juan Maldacena: While primarily known for his work on AdS/CFT correspondence, he has also contributed to understanding the holographic
 principle and its relation to quantum gravity.

These researchers have made significant contributions to our understanding of quantum gravity and its connection to loop theory. However, it's
 worth noting that QLG is an active area of research, and many other physicists are also working on related topics.
>
```

4) Connecting to LM Studio’s Local API from Python

Project 4 Task:

- Create an AI Assistant in Python and chat with it using an LLM from LM Studio
  - Keep responding to input questions with well-reasoned answers that "are both correct and helpful”
- Copy the ai.assistant example code from LM Studio to a python script & run it.

```python
# Connecting to LM Studio’s Local API
import requests
import json

url = "http://127.0.0.1:1234/v1/chat/completions"
headers = {"Content-Type": "application/json"}
data = {
    "model": "qwen/qwen3-1.7b",
    "messages": [
        {"role": "system", "content": "Always answer in rhymes."},
        {"role": "user", "content": "hey, can you tell me a joke?"},
    ],
    "temperature": 0.7,
    "max_tokens": -1,
    "stream": False
}

try:
    response = requests.post(url, headers=headers, data=json.dumps(data), stream=True)  # Added stream=True here too!
    response.raise_for_status() # Raise HTTPError for bad responses (4xx or 5xx)

    print(response.json()["choices"][0]["message"]["content"])

except requests.exceptions.RequestException as e:
    print(f"An error occurred: {e}")
```

LM Studio’s Best Practices

- **Best Practices & Tips**
  - Use the **built-in model search** to find compatible models.
  - Try different quantized models for better performance on low-end hardware. Supports GGUF models for efficient performance.
  - **Exporting:** Save your chat logs and prompts for later analysis or sharing.
  - Perfect for **quick demos** or sharing LLMs with non-technical friends.

A

- A

LLaMa.cpp - Installing

What is LLaMa.cpp?

- llama.cpp is a lightweight C++ implementation for running Llama and compatible models on CPU (and GPU).
  - It's great for older hardware or if you want to script everything. Maximum control, minimal overhead.
- **Download**: For most users, using a pre-built package is the easiest way to get started.
  - Windows: winget install llama.cpp
  - macOS: brew install llama.cpp
  - Linux (via Linuxbrew): brew install llama.cpp
  - The package is automatically updated with new llama.cpp releases.

```
# LLaMa.cpp
# Use a local model file
llama-cli -m ./models/llama-2-7b.ggmlv3.q4_0.bin -p "Hello, world!"

# Or download and run a model directly from Hugging Face
llama-cli -hf ggml-org/gemma-3-1b-it-GGUF
```

- **Pick & Download a Model:**
  - Browse Hugging Face and download a quantized .gguf model (e.g., Llama 7B).
- **Launch OpenAI-compatible API server:**
  Start the server with this command:

```
# Launch OpenAI-compatible API server
llama-server -hf ggml-org/gemma-3-1b-it-GGUF
```

The web server is running at http://127.0.0.1:6589/. Copy this URL and paste it into your browser to access the llama.cpp web interface.



Best Practices & Tips:

- Use quantized models for better performance on low RAM.
- Explore GPU acceleration if you have a compatible card (see the repo's README).
- Script batch tasks for automation.

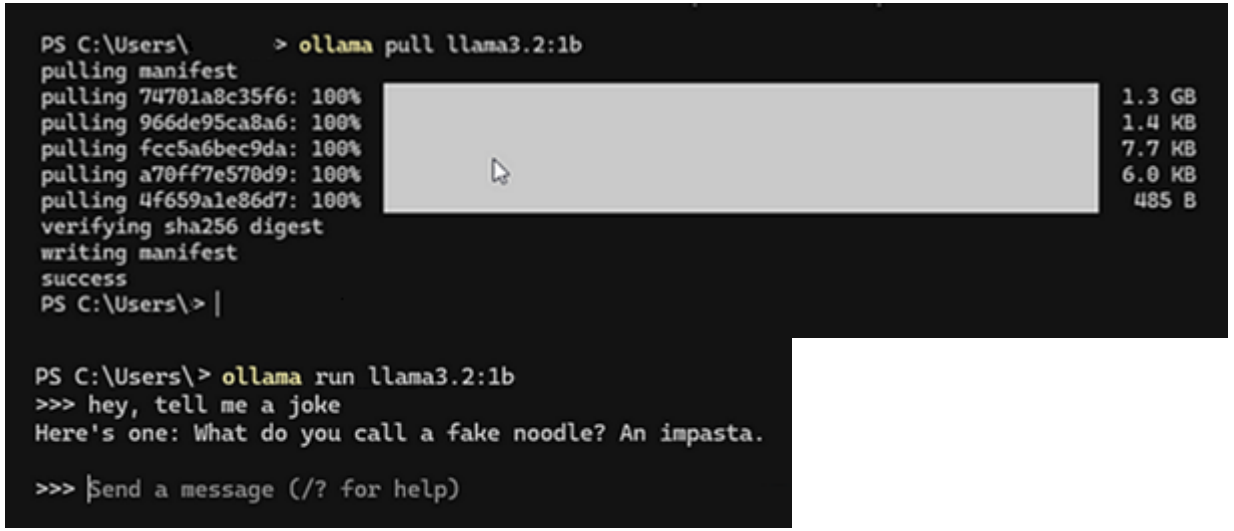
A

- A

Ollama - Installing

What is Ollama?

- **Ollama** is like the “App Store” for open-source LLMs. It gives you a simple command-line interface (CLI) to download, run, and manage models with almost zero setup!
  - The set-up is lightning-fast, supports both CLI and REST API, and integrates LLMs into your own apps with an ease.
- **Download**: Go to ollama.com and grab the installer for your OS.
- **Pick a Model:**
  Explore the available options in the **Ollama model library**.
- **Download the Model:** ollama pull llama3.2:1b
  - **Run the Model:** ollama run llama3.2:1b
  - **To see installed models:** ollama list
  - **To remove a model:** ollama rm llama3.2:1b



Ollama & Python



To Install Python client: pip install ollama

- **Sample code:**

```python
# Ollama Sample code
import ollama

response = ollama.chat(model='llama3.2:1b', messages=[{'role': 'user', 'content': 'Hello!'}])
print(response['message']['content'])
```

- **Use the REST API**
  - Ollama provides a REST API (Python example) for more advanced integration into your own scripts and applications.

```python
# Ollama REST API
import requests

response = requests.post(
    "http://localhost:11434/api/generate",
    json={"model": "llama3.2:1b", "prompt": "Tell me a joke about data science.", "stream":False}
)
print(response.json()["response"])
```

- **Best Practices & Tips**
  - Use ollama serve to run Ollama as a local API for your own apps.
  - **Turn off autostart** if you want to save system resources when not using Ollama.
  - **Custom Models:** You can import your own models or fine-tuned versions. Just follow the Ollama model import guide.
  - Check Ollama Docs for troubleshooting and advanced features.
  - Integrate with Open WebUI for a more advanced browser interface.

A

- A